

Top 10 Likely Interview Questions

Luxair Software Developer - Integration prep, concise version

Focus: Mule to Node.js/Express, integration patterns, API design, production debugging, SQL, resilience, and Node.js. Azure is treated as management/governance only, not as the runtime platform.

#	Question area	What to show
1	How would you migrate a Mule flow to Node.js/Express safely?	Mule -> Node.js migration, risk reduction, behavior compatibility.
2	What is the difference between SOA and microservices?	Architecture trade-offs, not buzzwords.
3	What does an ESB do, and why can it become a bottleneck?	Mule/ESB understanding, central integration layer risk.
4	How do you design a production-ready REST API?	REST beyond endpoint naming.
5	A POST /bookings request times out and the client retries. How do you avoid duplicate bookings?	Idempotency, retries, duplicate prevention.
6	One downstream service becomes slow. How do you prevent the whole system from degrading?	Timeout, retry, circuit breaker, bulkhead, backpressure, graceful degradation.
7	Sync or async: you need to notify five systems after a booking is created. What would you do?	Critical path vs async fan-out.
8	How would you debug a slow production API endpoint?	Production debugging and observability.
9	How would you optimize a slow SQL query?	SQL query optimization, not only indexing.
10	Why is Node.js a good fit for integration services, and where is it weak?	Node.js as an integration platform.

1. How would you migrate a Mule flow to Node.js/Express safely?

What they test: Mule -> Node.js migration, risk reduction, behavior compatibility.

Possible answer:

First I would understand the existing Mule flow, including routing, transformations, retries, error handling, timeouts, and edge cases. Then I would migrate incrementally, starting with low-risk flows, keeping old and new behavior comparable. I would use tests, logging, monitoring, and a rollback plan. I would not treat it only as code translation, because Mule often contains hidden integration behavior.

Reminder: Magyar emlékeztető: nem rewrite-big-bang, hanem incremental migration + rollback + behavior parity.

2. What is the difference between SOA and microservices?

What they test: Architecture trade-offs, not buzzwords.

Possible answer:

SOA and microservices both split systems into services, but traditional SOA often has more central governance and may use an ESB for routing, transformation, and orchestration. Microservices usually aim for smaller, independently deployable services with stronger data ownership and less central orchestration. I would not choose microservices automatically. I would look at team boundaries, deployment pain, data ownership, and operational maturity.

Reminder: Kulcs: SOA = service-oriented architecture. ESB gyakori SOA környezetben, de nem maga a SOA.

3. What does an ESB do, and why can it become a bottleneck?

What they test: Mule/ESB understanding, central integration layer risk.

Possible answer:

An ESB connects systems and can handle routing, transformation, protocol conversion, orchestration, retries, error handling, connectors, and monitoring. It can scale horizontally, but it can become a technical and organisational bottleneck if too many flows, transformations, and business rules accumulate in one central layer.

Reminder: Emlékeztető: API Gateway általában vékonyabb. ESB-ben gyakran több transformation/orchestration/business flow van.

4. How do you design a production-ready REST API?

What they test: REST beyond endpoint naming.

Possible answer:

I would focus on clear resource naming, correct HTTP methods and status codes, validation, structured errors, authentication and authorization, versioning, pagination, idempotency for retryable operations, rate limiting, and observability with request IDs and logs. For public APIs, backward compatibility and clear documentation are especially important.

Reminder: Szavak: stateless, 401 vs 403, 409 vs 422, structured error, request ID, OpenAPI.

5. A POST /bookings request times out and the client retries. How do you avoid duplicate bookings?

What they test: Idempotency, retries, duplicate prevention.

Possible answer:

I would use an idempotency key. The client sends a unique key with the request, and the server stores the key together with the result of the first processing attempt. If the same key is received again, the server returns the original result instead of creating a second booking. This is especially important for booking, payment, or order-like operations.

Reminder: Kapcsolódó: nonce/replay protection inkább security, idempotency inkább business correctness.

6. One downstream service becomes slow. How do you prevent the whole system from degrading?

What they test: Timeout, retry, circuit breaker, bulkhead, backpressure, graceful degradation.

Possible answer:

I would use explicit timeouts on outbound calls, limited retries with backoff and jitter, a circuit breaker to stop calling an unhealthy dependency, and bulkheads or per-downstream concurrency limits so one slow system cannot consume all resources. If the dependency is non-critical, I would degrade gracefully, for example by queuing the work or returning a partial result.

Reminder: Röviden: timeout = ne várj örökké. Circuit breaker = ne hívd tovább. Bulkhead = izoláld az erőforrást.

7. Sync or async: you need to notify five systems after a booking is created. What would you do?

What they test: Critical path vs async fan-out.

Possible answer:

I would keep only critical steps synchronous. For example, payment authorization may need to stay in the main request path. But notifications to CRM, email, loyalty, reporting, or other systems should usually be asynchronous through events or a queue. This reduces latency and avoids coupling the booking API availability to five downstream systems.

Reminder: Kérdés: mi kell azonnal a user response-hoz, és mi mehet később event/queue alapon?

8. How would you debug a slow production API endpoint?

What they test: Production debugging and observability.

Possible answer:

First I would check whether the problem affects one endpoint or the whole service. Then I would look at recent deployments, logs, metrics, and traces. I would check RED metrics: rate, errors, duration. Then I would inspect dependencies: database query latency, pool wait time, downstream calls, queue lag, CPU, memory, and event loop lag. I would use correlation IDs to follow one request across services.

Reminder: RED = Rate, Errors, Duration. USE = Utilization, Saturation, Errors. Nézd a p95/p99 latencyt is.

9. How would you optimize a slow SQL query?

What they test: SQL query optimization, not only indexing.

Possible answer:

First I would confirm the query is the bottleneck using traces or slow query logs. Then I would check the execution plan to see if there is a full table scan, bad join, missing index, expensive sort, or N+1 query problem. Possible fixes include better indexes, composite indexes, rewriting the query, selecting fewer columns, batching instead of N+1, keyset pagination, caching, or materialized views for expensive aggregates.

Reminder: Emlékeztető: index = külön keresőstruktúra. Composite index = több oszlopos index. GROUP BY = aggregáció.

10. Why is Node.js a good fit for integration services, and where is it weak?

What they test: Node.js as an integration platform.

Possible answer:

Node.js is a good fit for integration services because many integration workloads are I/O-bound: calling APIs, databases, queues, Redis, or external systems. Its async model handles many concurrent waiting operations efficiently. The weakness is CPU-heavy synchronous work, which can block the event loop. For that I would use worker threads, separate services, queues, or move the CPU-heavy work out of the request path.

Reminder: I/O = külső erőforrásra várás. CPU-bound = a processzor dolgozik intenzíven.

Azure adjustment

Azure note: based on your clarification, I would not expect deep Azure runtime questions. Keep Azure mostly at management/governance level: API Management, Key Vault/secrets, identity/access, monitoring/Application Insights, and CI/CD governance. I would not over-prepare AKS, Azure Functions, Container Apps, VM sizing, or deep Azure networking unless they specifically ask.

Fast final reminder

The strongest three themes to rehearse are: 1) Mule to Node.js migration, 2) downstream failure handling with timeout, retry, circuit breaker and bulkhead, 3) production debugging with logs, metrics, traces, database and downstream bottlenecks.

One sentence to keep in mind: do not answer with a technology-first mindset. Start with the problem, the risk, the trade-off, and then the implementation pattern.